Haskellog
A Prolog implementation in Haskell

Nicolas Dassas-Henny, Dionysis Yusofy, Mees Kerssens

Sunday 29th March, 2026

Contents

1 An Introduction to Haskellog **3**
1.1 Goals . . . 3
1.2 Running Haskellog . . . 4

2 Basic Types **4**

3 Prolog Pretty Printer **4**

4 Rule Parser **6**
4.1 Syntax . . . 6
4.2 Implementation of "or" and "and" . . . 7
4.3 Query Parser . . . 7

5 Unify **8**

6 Resolution **9**

7 Terminal User Interface **11**
7.1 Rendering . . . 12
7.2 Event handling . . . 12
7.3 Commands . . . 14
7.4 App definition . . . 14
7.5 Entry point . . . 15

8 Testing **15**

8.1 Parsers tests 15
8.2 Resolution tests 15

9 Conclusion **16**

10 appendix **17**

Bibliography **19**

1 An Introduction to Haskellog

1.1 Goals

Haskellog is an interpreter for a logic programming language similar to Prolog. The user gives a semantic meaning to some functions, and the program can then answer queries: it can determine whether a certain formula can be derived from the specified rules and, if it can, provide an instantiation of the variables that makes it derivable. If there are many such instantiations, Haskellog will provide up to 20 possibilities. Variables made be instantiated as constants but also as variables (for example if you define equality and query `X=X`, you could get the answer `X=X_12` which means that this is true for X being a variable which name here is X_12.)

The program expects a file containing logical rules made from terms. Terms are made with functions in lower case which can represent logical functions or logical predicates, variables in upper case, constants, which are simply functions of arity 0, and the logical connectors | (disjunction) and & (conjunction). For the sake of readability, in the following examples we use integers as syntactic sugar for Peano arithmetic (1 is `s(z)`, 2 is `s(s(z))`...) Each rule is either a fact, which is a single term like `plus(X,z,X).` that you could interpret as $\forall X \in \mathbb{N}, X + 0 = X$ or a rule composed of a conclusion and some premises, such as `sum_of_consecutives(N) :- plus(A, 1, B) & plus(A,B,N).` which could be interpreted as "a number N is a sum of two consecutive numbers A and B if $A + B = N$ and $A + 1 = B$". Obviously these interpretations are only given by the rules: the program does not know what `plus` means to you and if you define it, it is your responsibility to provide rules that match your intended meaning of each predicate.
For example, we can enrich the semantics of `plus` that we started defining before so that it matches the idea that `plus(X,Y,Z)` should be derivable if $X + Y = Z$ by adding the rule `plus(X,s(Y),s(Z)) :- plus(X,Y,Z).`.

The program can then answer queries, which are a succession of terms interpreted as a conjunction. Continuing with the previous example, a query could be `?- plus(X,Y,4), plus(X,2,Y)` which receives a positive answer from the program: `yes, X = 1 and Y = 3`. Once Haskellog has output one possible answer, the user can ask for additional ones.
You might think that the query `?- plus(X,Y,4), plus(X,1,Y)` would be given a similar answer `yes, X = 1,5 and Y = 2,5` but remember that we never defined what 1,5 or 2,5 are, the semantics only talks about integers. Therefore, the answer would be `no`. Try implementing half-integers to change that!

Internally, Haskellog works as follows: a parser reads the source file and transforms the rules and queries in some Haskell data types. Then, when a query is inputed in the UI, it is parsed and the prover attempts to solve it by resolution: the query is viewed as a conjunction of goals, and Haskellog repeatedly selects one goal and tries to match it with the head of a fact or rule from the input file. This matching is performed by unification, which computes a substitution for variables making two terms identical when possible. If no such substitution exists, that branch of the search fails and the program backtrack. When a rule matches, its premises replace the resolved goal, and the process continues recursively until either all goals have been solved, yielding a successful proof and an answer substitution, or no rule applies. The result of the proof is then presented to the user.

1.2 Running Haskellog

To build and run Haskellog run:

```
stack build
stack exec haskellog -- examples/family_relations.pl
```

Multiple rule files can be passed at once; their clauses are concatenated into a single program. Once the TUI opens, type a query in the input box and press `Enter`. For example, with `family_relations.pl` loaded, the query `ancestor(george, X).` will return all descendants of George. Press `Esc` or `Ctrl-C` to exit. Type `:help` to see additional built-in commands.

2 Basic Types

This module defines our basic types as well as some basic operations on them.

```haskell
module Terms where

import Test.QuickCheck hiding (Fun)
import Control.Monad (replicateM)
import Data.Map (Map)
import qualified Data.Set as Set
import Data.List (intercalate)

type Variable = String -- A variable is just defined by its name

data Term = Var Variable | Fun String [Term] -- A term is either a variable or a function (
    a 0-ary function being a constant)
  deriving (Eq,Ord,Show)

data Clause = Fact Term | Rule Term [Term] -- A Clause is either an axiom or a conclusion
    and a set of premisses
  deriving (Eq,Ord,Show)

type Program = [Clause] -- A program is a set of clauses

type Query = [Term] -- A query is a set of terms, interpreted as a conjunction

type Subst = Map Variable Term -- A substitution associates a term to a variable

varsTerm :: Term -> Set.Set Variable -- output a set of all variables in a term. Usefull
    for renaming them to fresh variables.
varsTerm (Var x) = Set.singleton x
varsTerm (Fun _ l) = Set.unions (map varsTerm l)

varsClause :: Clause -> Set.Set Variable -- idem for clauses
varsClause (Fact t) = varsTerm t
varsClause (Rule t l) = Set.unions (varsTerm t : map varsTerm l)

headClause :: Clause -> Term -- Fact/Conclusion
headClause (Fact t) = t
headClause (Rule t _) = t

bodyClause :: Clause -> [Term] -- Premises
bodyClause (Fact _) = []
bodyClause (Rule _ l) = l
```

3 Prolog Pretty Printer

These functions render our data structures as valid Prolog source text.

```haskell
-- | Render a 'Term' in Prolog notation.
prologTerm :: Term -> String
prologTerm (Var x)        = x
prologTerm (Fun name []) = name
prologTerm (Fun name args) =
  name ++ "(" ++ intercalate "," (map prologTerm args) ++ ")"

-- | Render a 'Clause' (facts end with *.*, rules use *:-*).
prologClause :: Clause -> String
prologClause (Fact t)       = prologTerm t ++ "."
prologClause (Rule h body) =
  prologTerm h ++ " :-\n    " ++
  intercalate " &\n    " (map prologTerm body) ++ "."

-- | Render a 'Program' as a Prolog source file.
prologProgram :: Program -> String
prologProgram = unlines . map prologClause

-- | Render a 'Query' in *?-* directive style.
prologQuery :: Query -> String
prologQuery []  = "?- true."
prologQuery ts  = "?- " ++ intercalate ", " (map prologTerm ts) ++ "."
```

The following code defines `Arbitrary` instances and generators for random `Term`s and `Clause`s. Originally, the intention was to use these with QuickCheck to test Haskellog against an existing Prolog interpreter written in Haskell. Unfortunately this turned out not to be feasible: the only available package did not support our version of GHC. Instead, we use the generators to verify that our parsers are correct: a randomly generated program is pretty-printed and parsed back, and we check that the result contains the original clauses (see Section on parser tests).

```haskell
genVar :: Gen Term
genVar = Var <$> elements ["X","Y","Z","W"]

genFunName :: Gen String
genFunName = elements ["a","b","c","f","g","h"]

genTerm :: Int -> Gen Term
genTerm 0 = oneof [genVar, Fun <$> genFunName <*> pure []]
genTerm n = oneof
  [ genVar
  , do name  <- genFunName
       arity <- choose (0, 3)
       args  <- replicateM arity (genTerm (n `div` 2))
       return (Fun name args)
  ]

instance Arbitrary Term where
  arbitrary = sized genTerm
  shrink (Var _)       = []
  shrink (Fun f args) = args ++ [Fun f as | as <- shrinkList shrink args]

instance Arbitrary Clause where
  arbitrary = oneof
    [ Fact <$> arbitrary
    , Rule <$> arbitrary <*> resize 3 (listOf1 arbitrary)
    ]
  shrink (Fact t)     = Fact <$> shrink t
  shrink (Rule h bs) = [Fact h]
                     ++ [Rule h' bs | h' <- shrink h]
                     ++ [Rule h bs' | bs' <- shrinkList shrink bs, not (null bs')]
```

4 Rule Parser

The lexer and parser for the rule files was created with happy and alex, and can be read in the appendix. We will focus here on the most interesting parts.

4.1 Syntax

Everyone can write their own text, string or Prolog files to create rules and make queries about. For this we explain how to formulate rules that can be interpreted by Haskellog:
Every rule consists of terms that can be either variables, functions or constants. Those can be represented by a string consisting of Latin letters, Western Arabic numerals and underscores. Variables have to to start with a capitalized letter and, constants and functions have to begin with a lower case letter. Function have to be followed by open bracket list of terms separated by `,` and a closed bracket. Since constants are interpreted as 0-ary function, one can also add brackets after them. Here a few examples:

- Varibles: `X`, `Variable`, `Z\_12`
- functions: `f(X,Y)`, `myFunction(myOtherFunction(X),Variable)`, `f\_34(f\_4(constant))`
- constant: `c`, `constant\_34`, `myConstant()`

Every rule consists of a conclusion (term) and conditions (also terms) separated by `\&`. In between the conclusion and the conditions we write `:-` as a symbol for "if". The rules are separated by `.`. If the conclusion holds without conditions, we can just follow the condition with a `.` or write `:-.`. Here an examples for a rule file[Wik04]:

```
parent(alice, bob).
parent(alice, charlie).
parent(bob, diana).
parent(charlie, emma).
parent(charlie, fred).
parent(george, alice).

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z) & ancestor(Z, Y).

sibling(X, Y) :- parent(P, X) & parent(P, Y).
```

We can also use | interpreted as an "or". Note that conjunction is stronger binding then disjunctions. So the example above can be written as:

```
parent(alice, bob):-.
parent(alice, charlie):-.
parent(bob, diana):-.
parent(charlie, emma):-.
parent(charlie, fred):-.
parent(george, alice):-.
```

```
ancestor(X, Y) :- parent(X, Y) | parent(X, Z) & ancestor(Z, Y).
sibling(X, Y) :- parent(P, X) & parent(P, Y).
```

4.2 Implementation of "or" and "and"

In the beta version it was not yet possible to use "or" as a connector of terms. We found a quick way to implement this using the infrastructure already given by Haskellog: With the following lines

```
Program
:                        {Right [Rule (Fun "_and" [Var "_X",Var "_Y"])
                         [Var  "_X",Var "_Y"],Rule (Fun "_or" [Var "_X",
                         Var "_Y"]) [Var "_X"],
                         Rule (Fun "_or" [Var "_X",Var "_Y"]) [Var "_Y"]]}
| Clause '.' Program    { case $3 of
                             Left e  -> Left e
                             Right xs -> Right ($1 : xs)
                           }
```

We insure that at the end of each set of rules the axioms of "and" and "or" are added has rules (in the sense of or program). The axioms are namely

```
and(X,Y):- X & Y.
or(X,Y):- X.
or(X,Y):- Y.
```

In our implementation of these rules we also added underscores before each name of variable or function to insure that a user is not further limited in naming terms. Note that the user can not define terms that start with a underscore via the normal parsing process.

4.3 Query Parser

When the user submits a query in the TUI text box, it is parsed by this module before being passed to the resolution engine. It is implemented with Parsec and accepts the same term syntax as the rule parser. The `?-` prefix and trailing `.` are optional, so both Prolog-style and bare queries are accepted. Conjunction (`&`) and disjunction (`|`) are supported, with conjunction binding more tightly.

```haskell
module QueryParser (Query, parseQuery) where

import Text.Parsec
import Text.Parsec.String (Parser)
import Terms

ws :: Parser ()
ws = spaces

lexeme :: Parser a -> Parser a
lexeme p = p <* ws

sym :: String -> Parser String
```

```
sym s = lexeme (string s)

pVar :: Parser Term
pVar = lexeme $ do
  first <- upper <|> char '_'
  rest  <- many (alphaNum <|> char '_')
  return $ Var (first : rest)

pAtom :: Parser Term
pAtom = lexeme $ do
  first <- lower
  rest  <- many (alphaNum <|> char '_')
  return $ Fun (first : rest) []

pStruct :: Parser Term
pStruct = lexeme $ do
  first <- lower
  rest  <- many (alphaNum <|> char '_')
  let name = first : rest
  args <- between (sym "(") (sym ")") (pTerm `sepBy` sym ",")
  return $ Fun name args

pTerm :: Parser Term
pTerm = try pStruct <|> try pVar <|> pAtom

pQuery :: Parser Query
pQuery = do
  ws
  optional (try (sym "?-"))
  expr <- pOr
  optional (sym ".")
  eof
  return [expr]

pOr :: Parser Term
pOr = chainl1 pAnd (sym "|" >> return (\a b -> Fun "_or" [a, b]))

pAnd :: Parser Term
pAnd = chainl1 pTerm (sym "&" >> return (\a b -> Fun "_and" [a, b]))

parseQuery :: String -> Either ParseError Query
parseQuery = parse pQuery "<query>"
```

5 Unify

This module implement unification as well as a few other helpfull functions regarding unification/substitution. Unifying two terms mean finding variables substitutions such that the two terms are equal. It is used later to find clauses that match a certain goal.

```
module Unify where

import qualified Data.Map as Map
import Terms
```

applySubst substitutes every variables in a term with their associated value.

```
applySubst :: Subst -> Term -> Term
applySubst s (Var x) =
  case Map.lookup x s of
    Nothing -> Var x
    Just t -> applySubst s t -- we propagate the substitution
applySubst s (Fun name l) =
  Fun name (map (applySubst s) l)
```

occurencechek checks if the name of a variable appears in a term. If it's the case we can't perform unification. If we say that `X` is now `f(X)` we get that it is also equal to `f(f(x))`. `f` does not have a predefined semantic, this is thuse false.

```
occurrenceCheck :: String -> Term -> Bool
occurrenceCheck x (Var y) = x == y
occurrenceCheck x (Fun _ l) = any (occurrenceCheck x) l
```

unification returns a substitution if it finds one that unify two terms under a predefined substitution (that will be empty at the start of our program).

```
unification :: Subst -> Term -> Term -> Maybe Subst
unification sub t1 t2 =
  case (applySubst sub t1, applySubst sub t2) of
    (Var x, Var y) ->
      if x == y then Just sub else Just (Map.insert x (Var y) sub)
    (Var x, fun) ->
      if occurrenceCheck x fun then Nothing else -- The variable appears in the function,
          we can't perform unification !
        Just (Map.insert x fun sub)
    (fun, Var y) ->
      if occurrenceCheck y fun then Nothing else -- idem
        Just (Map.insert y fun sub)
    (Fun name1 l1, Fun name2 l2)
      | name1 /= name2 -> Nothing
      | otherwise -> unifyLists sub l1 l2

unifyLists :: Subst -> [Term] -> [Term] -> Maybe Subst
unifyLists sub [] [] = Just sub
unifyLists sub (x1:xs1) (x2:xs2) =
  unification sub x1 x2 >>= (\ s -> unifyLists s xs1 xs2)
unifyLists _ _ _ = Nothing

applySubstClause :: Subst -> Clause -> Clause
applySubstClause sub (Fact t) = Fact (applySubst sub t)
applySubstClause sub (Rule t l) = Rule (applySubst sub t) (map (applySubst sub) l)

applySubstQuery :: Subst -> Query -> Query
applySubstQuery sub = map (applySubst sub)
```

6 Resolution

This module is about the resolution algorithm which is the heart of Haskellog. It works as follow:

- Select a query to solve
- Find a rule or fact whose head can be unified with this query, using fresh variables
- Apply the unification and extend the current substitution
- If the rule is a fact, the query is solved; otherwise, replace the query with the body of the rule (possibly several new queries)
- If a step fails (no unification possible or a later query cannot be solved), backtrack to try another rule or fact
- When all queries are solved, return the final substitution

```haskell
module Resolution where

import Unify
import Terms
import Control.Monad.State
import qualified Data.Map as Map
import qualified Data.Set as Set
import Data.List (nub)
import Data.Maybe (catMaybes)
```

We will need in our algorithm to rename some variables to fresh one because the application of a rule is supposed to be local, Haskellog might apply the same rule twice but on different variables. For that we use the state monad incrementing an integer each time we want to reuse a variable (we first use the variable X1 then X2...).

From X, freshVarName returns `State X\_n (n+1)` with n the internal state of the monad.

```haskell
freshVarName :: Variable -> State Int Variable
freshVarName x = do
  n <- get
  put (n + 1)
  return (x ++ "_" ++ show n)
```

buildRenaming takes a set of Variables and return a renaming associating them to fresh variables.

```haskell
buildRenaming :: Set.Set Variable -> State Int (Map.Map Variable Variable)
buildRenaming vars = do
  pairs <- mapM freshPair (Set.toList vars)
  return (Map.fromList pairs)
  where
    freshPair x = do
      y <- freshVarName x
      return (x, y)
```

renameClause applies the renaming of terms inside a clause.

```haskell
renameTerm :: Map.Map String String -> Term -> Term
renameTerm env (Var x) =
  case Map.lookup x env of
    Just y  -> Var y
    Nothing -> Var x
renameTerm env (Fun f args) =
  Fun f (map (renameTerm env) args)

renameClause :: Map.Map String String -> Clause -> Clause
renameClause env (Fact t) =
  Fact (renameTerm env t)
renameClause env (Rule h body) =
  Rule (renameTerm env h) (map (renameTerm env) body)

freshenClause :: Clause -> State Int Clause
freshenClause clause = do
  let vars = varsClause clause
  env <- buildRenaming vars
  return (renameClause env clause)
```

We can now write "do clause' <- freshenClause clause" to get a clause with fresh variable in clause'.

`solveOneClause` is one step of the SLD resolution algorithm, we take the current state of our substitution, one clause of the program which we will replace variables by fresh ones, one goal of the query, the rest of the query and we try to unify the term with the clause. If we suceed, we return the substitution that worked as well as the queries that still need to be computed.

solveGoal give a list of the substitutions coming from successful unifications with clauses of our program.

```
solveOneClause :: Subst -> Clause -> Term -> Query -> State Int (Maybe (Subst, Query))
solveOneClause initSub clause goal restQuery =
  do
    clause' <- freshenClause clause
    case unification initSub goal (headClause clause') of
      Nothing -> pure Nothing
      Just sub ->
        let query = applySubstQuery sub (bodyClause clause' ++ restQuery) in
        pure (Just (sub, query))

solveGoal :: Subst -> Program -> Term -> Query -> State Int [ (Subst, Query) ]
solveGoal initSub prog goal restQuery = do
  fmap catMaybes (mapM (\clause -> solveOneClause initSub clause goal restQuery) prog)
```

This is the main algorithm performing SLD resolution. In entry we take our programs which is a list of clauses, a substitution which should be empty as first but will be useful for recursive calls, as well as a list of our goal, the program should output a (possibly infinite) list of substitutions of the variables such that the conjonctive query is made true. The idea is just to aly recursively the step we defined earlier.

```
resolve :: Subst -> Program -> Query -> State Int [Subst]
resolve sub _ [] = pure [sub]
resolve currentSub prog (goal:restGoals) = do
  nextSteps <- solveGoal currentSub prog goal restGoals
  concat <$> mapM (\(sub', query') -> resolve sub' prog query') nextSteps
```

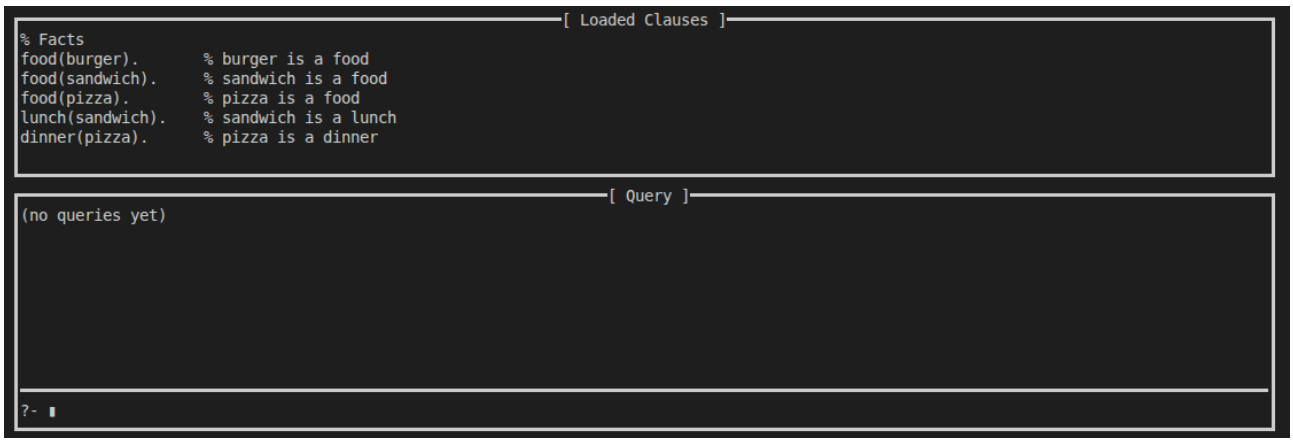
We now define the clean resolution, hiding the mechanism using the state monad or the explicit "current substitution" in argument. We just start the resolve programm with an empty substitution and with the state 0. We also remove all the internal variables from the computation and normalize them by aplying the substitution as far as we can (to get X = bob Y = bob instead of X=Y Y=bob for example).

```
resolution :: Program -> Query -> [Subst]
resolution prog query =
  let solutions = evalState (resolve Map.empty prog query) 0
  in nub (map (restrictSubst query) solutions)

restrictSubst :: Query -> Subst -> Subst
restrictSubst query sub =
  let vars = Set.unions (map varsTerm query) -- all the variables in our query
  in Map.fromList
       [ (x, applySubst sub t) -- we do the normalization here by alying applySubst
       | (x, t) <- Map.toList sub
       , x `Set.member` vars -- we only keep variables that are in the input query
       ]
```

7 Terminal User Interface

The TUI is implemented using the Brick library and is the main way to interact with Haskellog. It consists of two panes: the top pane shows the clauses loaded from the rule files, and the bottom pane is the interactive query interface.



The user types a query into the input box and presses `Enter`. Haskellog parses the query, runs the algorithm and prints the resulting variable bindings. If the query is ground and provable it returns `true.`, and if no proof exists it returns `false.`. Up to 20 solutions are shown.

The state of our TUI: the loaded file contents, the query history, and whatever the user is currently typing.

```haskell
data TuiState = TuiState
  { clausesText :: String
  , history     :: [String]
  , inputBuf    :: String
  }
```

7.1 Rendering

We draw two bordered panes stacked on top of each other.

```haskell
drawClausesPane :: TuiState -> Widget Name
drawClausesPane s =
  let content = if null (clausesText s)
                then str "(no file loaded)"
                else str (clausesText s)
  in withBorderStyle unicodeBold
     (borderWithLabel (str "[ Loaded Clauses ]")
     (viewport ClausesVP Vertical content))

drawQueryPane :: TuiState -> Widget Name
drawQueryPane s =
  let histWidgets = if null (history s)
                    then [str "(no queries yet)"]
                    else map str (history s)
      inputLine   = str "?- " <+> str (inputBuf s) <+> str "\9646"
  in withBorderStyle unicodeBold
     (borderWithLabel (str "[ Query ]")
     (vBox [viewport HistoryVP Vertical (vBox histWidgets), hBorder, inputLine]))

drawUI :: TuiState -> [Widget Name]
drawUI s = [vBox [vLimitPercent 40 (drawClausesPane s), drawQueryPane s]]
```

7.2 Event handling

Key presses are handled here: printable characters are appended to the input buffer, backspace removes the last character, Enter submits the query, and Esc or Ctrl-C exits the program.

```haskell
addDot :: String -> String
addDot q = if null q || last q == '.' then q else q ++ "."

queryVars :: [Term] -> [String]
queryVars terms = Set.toList (Set.unions (map varsTerm terms))

renderOneSolution :: [String] -> Subst -> String
renderOneSolution vars sub =
  intercalate ", "
    [ v ++ " = " ++ prologTerm (Map.findWithDefault (Var v) v sub)
    | v <- vars
    ]

renderSolutions :: [Term] -> [Subst] -> [String]
renderSolutions terms sols
  | null sols   = ["false."]
  | null vars   = ["true."]
  | otherwise   = map (renderOneSolution vars) sols
  where
    vars = queryVars terms

handleEvent :: BrickEvent Name () -> EventM Name TuiState ()

-- quit on Esc or Ctrl-C
handleEvent (VtyEvent (V.EvKey V.KEsc [])) = halt
handleEvent (VtyEvent (V.EvKey (V.KChar 'c') [V.MCtrl])) = halt

-- submit on Enter
handleEvent (VtyEvent (V.EvKey V.KEnter [])) = do
  s <- get
  let inp = inputBuf s
  newLines <- case parseCommand inp of
    Just cmd -> runCommand cmd
    Nothing  -> do
      let q       = addDot inp
          outLines = case parseQuery q of
                       Left err -> ["Query parse error: " ++ show err]
                       Right terms ->
                         case pProgram (clausesText s) of
                           Left err  -> ["Rule parse error: " ++ err]
                           Right prog -> renderSolutions terms (take 20 (resolution prog
                               terms))
      return (["?- " ++ q] ++ outLines ++ [""])
  modify (\st -> st { history = history st ++ newLines, inputBuf = "" })
  vScrollToEnd (viewportScroll HistoryVP)

-- backspace to delete character
handleEvent (VtyEvent (V.EvKey V.KBS [])) = do
  s <- get
  let buf = inputBuf s
  if null buf
    then return ()
    else modify (\st -> st { inputBuf = init buf })

-- typing characters
handleEvent (VtyEvent (V.EvKey (V.KChar c) [])) =
  modify (\s -> s { inputBuf = inputBuf s ++ [c] })

-- scroll the clauses pane
handleEvent (VtyEvent (V.EvKey V.KUp [])) =
  vScrollBy (viewportScroll ClausesVP) (-1)
handleEvent (VtyEvent (V.EvKey V.KDown [])) =
  vScrollBy (viewportScroll ClausesVP) 1

-- mouse scroll on whichever viewport the cursor is over
handleEvent (MouseDown n V.BScrollUp _ _) =
  vScrollBy (viewportScroll n) (-1)
handleEvent (MouseDown n V.BScrollDown _ _) =
  vScrollBy (viewportScroll n) 1

-- ignore everything else
handleEvent _ = return ()
```

7.3 Commands

The TUI supports colon-prefixed commands like GHCi. Typing `:help` lists them, `:gen term`, `:gen clause`, `:gen program` and `:gen query` use the QuickCheck generators and pretty-prints the result.

```
data Command = Help | GenTerm | GenClause | GenProgram Int | GenQuery Int

parseCommand :: String -> Maybe Command
parseCommand s = case words s of
  [":help"]                          -> Just Help
  [":gen", "term"]                   -> Just GenTerm
  [":gen", "clause"]                 -> Just GenClause
  [":gen", "program"]                -> Just (GenProgram 5)
  [":gen", "program", n] | [(k,"")] <- reads n -> Just (GenProgram k)
  [":gen", "query"]                  -> Just (GenQuery 3)
  [":gen", "query", n]   | [(k,"")] <- reads n -> Just (GenQuery k)
  _                                   -> Nothing

runCommand :: Command -> EventM Name TuiState [String]
runCommand Help = return
  [ ":help               - show this message"
  , ":gen term           - random term"
  , ":gen clause         - random clause"
  , ":gen program [n]    - random program (default 5 clauses)"
  , ":gen query [n]      - random query   (default 3 goals)"
  , ""
  ]
runCommand GenTerm = do
  t <- liftIO (generate arbitrary :: IO Term)
  return [prologTerm t, ""]
runCommand GenClause = do
  c <- liftIO (generate arbitrary :: IO Clause)
  return [prologClause c, ""]
runCommand (GenProgram n) = do
  cs <- liftIO (generate (vectorOf n (resize 4 arbitrary) :: Gen [Clause]))
  return (lines (prologProgram cs) ++ [""])
runCommand (GenQuery n) = do
  ts <- liftIO (generate (vectorOf n (resize 4 arbitrary) :: Gen [Term]))
  return [prologQuery ts, ""]
```

7.4 App definition

The app definition bundles together the drawing, event handling, cursor, and styling logic into a single Brick application value.

```
tuiApp :: App TuiState () Name
tuiApp = App
  { appDraw         = drawUI
  , appChooseCursor = showFirstCursor
  , appHandleEvent  = handleEvent
  , appStartEvent   = do
      vty <- getVtyHandle
      liftIO $ V.setMode (V.outputIface vty) V.Mouse True
  , appAttrMap      = const (attrMap V.defAttr [])
  }
```

7.5 Entry point

This code is run by the main function in Main.lhs. It initializes the TuiState with the loaded clauses and an empty history and input buffer, then runs the Brick application.

```
runTui :: String -> IO ()
runTui clauses = do
  let initState = TuiState
        { clausesText = clauses
        , history     = []
        , inputBuf    = ""
        }
  _ <- customMainWithDefaultVty Nothing tuiApp initState
  return ()
```

8 Testing

8.1 Parsers tests

We use the `Arbitrary` instances from `Terms` to generate random programs and check that the pretty-printer and parser are inverses of each other.

```
module Main where

import Test.Hspec
import Test.QuickCheck
import Terms
import ProgramParser (pProgram)
import QueryParser (parseQuery)

main :: IO ()
main = hspec $ do
  describe "ProgramParser" $
    it "parsed result contains all original clauses" $ property $
      \prog -> counterexample (prologProgram prog) $
        case pProgram (prologProgram prog) of
          Left err  -> counterexample err False
          Right got -> property $ all (`elem` got) prog
  describe "QueryParser" $
    it "parses any generated term without crashing" $ property $
      \t -> counterexample (prologTerm t) $
        case parseQuery (prologTerm t) of
          Left err -> counterexample (show err) False
          Right _  -> property True
```

8.2 Resolution tests

To test the resolution algorithm, we use a set of small fixed examples. Each test case is a directory under `test/cases/` containing a rule file `program.pl`, a query `query.txt`, and the expected output `expected.txt`. The test suite parses each program and query, runs the resolution engine, formats the answers, and compares the result to the expected file.

```
module Main where

import Control.Monad (forM_)
import Data.Char (isSpace)
import Data.List (dropWhileEnd, intercalate, sort)
import qualified Data.Map as Map
import qualified Data.Set as Set
```

```haskell
import ProgramParser (pProgram)
import QueryParser (parseQuery)
import Resolution (resolution)
import System.FilePath ((</>))
import Terms
import Test.Hspec

casesDir :: FilePath
casesDir = "test/cases"

caseNames :: [String]
caseNames = ["01_meal", "02_ground_yes", "03_ground_no", "04_ancestor", "05_empty",
             "06_peano", "07_multi_goal", "08_multiple_solutions"]

trim :: String -> String
trim = dropWhileEnd isSpace . dropWhile isSpace

renderAnswers :: Query -> [Subst] -> [String]
renderAnswers q sols
  | null sols = ["false."]
  | null vs   = ["true."]
  | otherwise = sort (map oneRow sols)
  where
    vs = sort (Set.toList (Set.unions (map varsTerm q)))
    oneRow sub = intercalate ", "
      [v ++ " = " ++ prologTerm (Map.findWithDefault (Var v) v sub) | v <- vs]

runCase :: String -> IO ()
runCase name = do
  let d = casesDir </> name
  progSrc  <- readFile (d </> "program.pl")
  qSrc     <- readFile (d </> "query.txt")
  expected <- sort . filter (not . null) . map (trim . filter (/= '\r')) . lines
                <$> readFile (d </> "expected.txt")
  case (pProgram progSrc, parseQuery (trim (head (lines qSrc)))) of
    (Left e, _)           -> expectationFailure ("program: " ++ e)
    (_, Left e)           -> expectationFailure ("query: " ++ show e)
    (Right prog, Right q) -> renderAnswers q (resolution prog q) `shouldBe` expected

main :: IO ()
main = hspec $
  describe "resolution, cases in test/cases/" $
    forM_ caseNames $ \n -> it n (runCase n)
```

9 Conclusion

Haskellog successfully imitates Prologs basic functionalities. Our QuickCheck tests show that the program at least doesn't crash when fed strings representing rules our quiries. With more time we would maybe add the possibility to negate formulas.
We originally planed to test our implementation with the an already existing Haskell implementation of Prolog[Bar20]. The problem was that this implementation runs on an older version of Haskell. Although we didn't find any bugs by simply coming up with queries and set of rules, we would be more secure in our product if we did some randomized testing of the statement it returns.
This project was great conclusion to the beginning of journey with functional programming. We were able to make use of the state monad and pattern matching, which were novel to us. We are looking forward to keep dabbling in functional programming in the future.

10 appendix

Alex input for Lexer

```
{
module RuleLexer where
import Terms
}
%wrapper "basic"

$highkey = [A-Z]
$lowkey = [a-z]
$restOfName = [a-zA-Z0-9_]
--$white = [ \n\t]

@varName = $highkey $restOfName*
@funName   = $lowkey $restOfName*

tokens :-

    $white+              ;
    \% [^\n]*             ;
    @varName            { \s -> VARNAME s }
    @funName            { \s -> FUNNAME s }
    \.                  { \_ -> ENDOFRULE}
    \,                  { \_ -> VARSEP}
    \&                  { \_ -> AND }
    ":-"                { \_ -> IF }
    \(                  { \_ -> LPAREN }
    \)                  { \_ -> RPAREN }
    \|                  { \_ -> OR}

{
data Token
  = FUNNAME String
  | VARNAME Variable
  | ENDOFRULE
  | LPAREN
  | RPAREN
  | VARSEP
  | AND
  | IF
  | OR
  deriving (Show, Eq)
}

-- written with the help of this article: https://blog.cofree.coffee/2021-10-29-happy-an
```

Happy input for Parser

```
{
module ProgramParser where
import RuleLexer
import Terms
import Unsafe.Coerce (unsafeCoerce)
}

%name parseProgram
%tokentype {Token}
%error {parseError}

%token
    funname {FUNNAME $$}
    varname {VARNAME $$}
    '.'     {ENDOFRULE}
    '('     {LPAREN}
    ')'     {RPAREN}
    ','     {VARSEP}
    ":-"    {IF}
    "|"     {OR}
    '&'     {AND}

%%

Program
    :                       {Right [Rule (Fun "_and" [Var "_X",Var "_Y"]) [Var "_X",Var
                            Rule (Fun "_or" [Var "_X",Var "_Y"]) [Var "_X"],
                            Rule (Fun "_or" [Var "_X",Var "_Y"]) [Var "_Y"]]}
    | Clause '.' Program    { case $3 of
                                Left e  -> Left e
                                Right xs -> Right ($1 : xs)
                              }


Clause
    : Term ":-"             {Fact $1}
    | Term                  {Fact $1}
    | Term ":-" Assum       {Rule $1 $3}

Assum
    : Conj              { $1 }
    | Assum2 "|" Conj1    {(Fun "_or" [$3,$1]):[]}
Assum2
    : Conj1              { $1 }
    | Assum2 "|" Conj1    {(Fun "_or" [$3,$1])}
```

```
Conj
    : ConjChild             { $1:[] }
    | ConjChild '&' Conj    {$1: $3}

Conj1
    : ConjChild1             { $1 }
    | Conj1 '&' ConjChild1    {Fun "_and" [$1,$3]}

ConjChild1
    : Term                  {$1}
    | '('Assum2')'           {$2}
ConjChild
    : Term                  {$1}
    | '('Assum2')'           {$2}
{-Assum
    : Term   { $1 : [] }
    | Term '&' Assum    { $1 : $3 }-}

Term
    : funname '(' Terms ')' {Fun $1 $3}
    | funname               {Fun $1 []}
    | varname               {Var $1}

Terms
    :                   {[]}
    | Term              {$1 :[]}
    | Term ',' Terms    {$1 : $3}

{
parseError :: [Token] -> a
parseError tokens =
  let ctx = take 8 tokens
      msg = "Parse error near tokens: " ++ show ctx
  in unsafeCoerce (Left (msg :: String) :: Either String [Clause])

pProgram:: String -> Either String [Clause]
pProgram = parseProgram . alexScanTokens}

-- written with the help of this article: https://blog.cofree.coffee/2021-10-29-happy-an
```

References

[Bar20] Matthias Bartsch. A prolog interpreter written in haskell., 2020.

[Wik04] Wikibooks. Prolog/recursive rules, 2004.